

Rapport Système Informatique:

Partie Compilateur :

Règle langage :

Notre fichier Yacc définit des règles de langage basique :

- Body : symbolise une zone récursive d'ensemble instructions
- Instruction : représente un type d'instructions on se trouve (déclaration, affectation, if/else zone, fonction appelée). L'instruction fait souvent appel à une expression ou un nouveau body dans le cas d'un if/else par exemple
- GroupeDecl : représente différents types de déclaration (constantes, pointeurs, vars etc). Elles permettent de déclarer un ou plusieurs espaces mémoire pour accueillir l'objet de la déclaration.
- Expression : permet de générer l'évaluation et le stockage d'une expression (une expression peut faire appel à elle-même du côté gauche, et à droite le type divmul)
- Divmul: permet d'évaluer les divisions et multiplications, ainsi que d'accéder à la dernière strate de la récursion : les termes.
- Terme: représente une valeur. Qu'elle soit par référence à une variable, appel de fonction, déréférencement de pointeur ou bien simplement une constante.

Compilation orientée mémoire :

Dans un premier temps, nous effectuons une compilation orientée mémoire. Cette compilation permet de fournir un code assembleur indiquant directement dans le code les adresses mémoire utilisées en tant qu'opérandes.

Pour gérer la mémoire, nous avons créé un fichier "gestionMem.c" s'occupant de l'attribution de l'espace mémoire. Ce fichier gère la table des symboles.

La table des symboles est une hashtable de nœuds. Le nombre de nœuds maximal est de 1024. Si on instancie un nœud de plus, le compilateur lève une erreur. Un nœud permet de représenter une variable, un pointeur, une constante. La clé utilisée par la table de hachage se base sur le nom de la variable. Pour simplifier la fonction de hachage, nous avons décidé de n'accepter que les noms de variable de longueur 4. Cela permet d'obtenir une fonction bijective du nom de la variable vers l'ensemble des entiers codés sur 32 bits.

```
int var_id_to_index(char var_name[5]){
    int index = 0;
    for(int i = 0; i < 4; i++){
        index |= var_name[i] << (i*4);
    }
    return index;
}
```

Pour éviter de réserver en mémoire une table des symboles de taille 2^{32} (nombre de valeur possible sur 32 bit), nous limitons sa taille via une opération modulo sur la clé de 32 bits. Cela enlève la bijection que l'on avait maintenue au profit d'une structure de données stockables en mémoire vive.

Pour gérer les collisions, notre table de hachage contient une liste. Quand on cherche dans la table, on vérifie que l'id est le même que celui de la variable recherchée. Sinon on se déplace dans la liste pour trouver le bon nœud.

La distribution des adresses mémoires se fait via la fonction `add_var` qui crée un nœud dans la table de hachage grâce à une variable global représentant la dernière adresse mémoire libre. Cette variable est incrémentée lors de la déclaration d'une nouvelle variable et décrémentée par la gestion du scope.

Gestion du scope :

Au fil de l'analyse du code d'entrée, un attribut de structure est utilisé pour représenter la profondeur d'appartenance d'une variable. Cette profondeur est utile pour reconnaître quelle variable est accessible dans quel scope (donc gérer les homonymes et les utilisations de variables inaccessibles). Connaître notre profondeur actuelle permet aussi de libérer de la mémoire en "oubliant" les variables internes au scope que l'on vient de quitter par décrémentation de l'adresse de dernière variable libre.

La variable observant la profondeur actuelle est incrémentée via un appel de fonction dans le fichier `yacc`. A chaque ouverture de crochet (`{}`), un nouveau corps est reconnu.

Libération Mémoire :

La libération de mémoire se fait à la fin de chaque body après la décrémentation du scope. Pour libérer la mémoire, chaque élément de la hashtable est analysé. Si l'élément à été défini dans un espace avec une profondeur plus grande que l'actuel, alors il est supprimé de la hashtable et `"derniere_addr_libre"` est décrémentée. Une analogie pourrait être de voir le comportement des adresses mémoire comme une pile : on empile de nouvelles adresses quand on rentre dans un body et on les dépile quand on en sort. Cette méthode nous permet d'assurer qu'il n'y ait pas de fuite de mémoire dès lors que toutes les variables manipulées le sont dans des corps bien définis.

Pointeurs :

Un pointeur est géré comme une variable, on l'ajoute à la table de symbole. La seule différence est qu'il est indiqué comme un pointeur dans la table de hachage. Cela permet de changer le comportement du compilateur lors de son utilisation : il faut pouvoir le déréférencer pour obtenir la valeur pointée par celui-ci via le symbole `"*"` (tMUL). *Le jeu d'instructions proposé par le sujet a dû être étendu car il ne supporte pas cette action. Nous avons donc introduit l'instruction "DEREF".*

Branchements conditionnels (if/else):

Pour gérer le if et le else, nous utilisons les instructions 7 et 8 qui sont respectivement le jump inconditionnel et conditionnel. La gestion du if avec else optionnelle se fait en 5 étapes. Pour gérer les if else, nous avons implémenté une pile qui empile les lignes assembleur incomplète. Celles-ci devront être modifiées a posteriori pour indiquer sur quel ligne effectuer un jump. Cette double passe est obligatoire car l'espace des numéros d'instructions évoluent au cours de la génération du code. On ne peut pas prévoir la taille d'un corps pour sauter par-dessus avant de l'avoir généré.

1. évaluation de l'expression du if : on écrit dans le fichier le code pour évaluer la condition
2. On sauvegarde la ligne de if. Pour cela on push la ligne actuelle du fichier. Dans le fichier de sortie on écrit un commentaire temporaire "début de if;"
3. On écrit le code assembleur du body
4. A la fin du body, on calcule l'adresse à laquelle on doit jump dans le cas où la condition n'est pas valide. Pour cela on fait un fseek_line (version de fseek basée sur les lignes comme unité et non les octets, écrite par nos soins.) du fichier + 2. Le + 2 permet de prévoir un potentiel "else".
5. On retourne à l'emplacement du fichier avec le if à l'aide d'un pop sur la pile des lignes "à terminer", on écrit la ligne pour jump à la bonne adresse et on revient à la fin du fichier.

2 cas sont alors possible :

- On parse un else. Dans ce cas là, la même structure que pour le if est reproduite avec un push puis pop. Cependant cette fois-ci le jump est inconditionnel. Ce jump permet de contourner le else dans le cas ou la condition du if est remplie
- On ne parse pas de else. Dans ce cas là, on génère un saut qui ne sert à rien pour compenser un offset de saut constant (car dans le if on fait un saut + 2).

Notre méthode de if/else est efficace grâce à sa pile qui permet de faire des if else imbriqués sans se perdre sur les retour de ligne. Le backtracking est facilité par la pile car elle garde en haut de pile l'adresse actuelle ou l'on peut calculer le saut.

Branchements conditionnels (while):

Le while fonctionne presque comme le if. A la fin du while on fait un saut inconditionnel pour remonter au début du while et recalculer l'expression. Au début du while on fait un saut conditionnel qui vérifie l'expression. Si la condition est fausse, on saute après le while.

Fonctions :

1. Appel de fonction (Call)

Comme en C, les fonctions doivent être définies avant le main. Le mécanisme d'appel repose sur une gestion de pile virtuelle que l'on a défini à partir de la dernière adresse 32 bit disponible (comme elle on décrémente les adresses pour l'étendre).

Réservation de l'adresse de retour :

Avant de traiter les arguments, on insère un placeholder dans le fichier assembleur. Comme les arguments peuvent être des expressions, on ne connaît l'adresse de retour qu'une fois l'appel totalement généré.

Gestion de la pile :

Pour chaque argument, la fonction `push_arg()` décrémente le pointeur de pile virtuel. On copie ensuite la valeur de l'argument sur la pile.

Retour post intro :

Une fois la séquence d'appel générée, on récupère le numéro de ligne actuel. On utilise `fseek_line` pour revenir au placeholder et écrire l'instruction de sauvegarde de l'adresse de retour à l'emplacement mémoire : $= \text{get_sp}() + \text{get_func_args}() + 1$

On termine l'appel par un saut inconditionnel vers la ligne de départ de la fonction, récupérée via le gestionnaire de fonctions.

2. Définition de fonction

Les fonctions possèdent leur propre table de symboles pour garantir l'isolation des variables.

Enregistrement dans la table :

Lors de la lecture de la déclaration, le nom de la fonction et sa ligne de départ sont stockés dans une table de hachage.

Passage des arguments :

Pour chaque argument déclaré, on crée une variable locale à la fonction. Des placeholders sont insérés pour les instructions de "pop".

Récupération des valeurs :

Une fois le nombre total d'arguments connu, le compilateur revient sur les placeholders pour générer les copies de la pile vers les adresses des variables locales. On simule ensuite le dépilage pour maintenir le `sp` cohérent.

Corps de la fonction :

Le code du corps est généré comme un corps classique en utilisant les variables locales scopées.

3. Retour et Fin de fonction

Valeur de retour :

Lorsqu'un token `return` est rencontré, le résultat de l'expression qui suit est copié à l'adresse mémoire globale `RESULT_MEM_ADDR`.

Saut de sortie (`JMPREF`) :

Pour quitter la fonction, on utilise l'instruction `JMPREF`. Elle lit l'adresse de retour stockée sur la pile (à l'indice du `sp` actuel si tout se passe bien) et effectue un saut vers cette ligne.

Cette instruction n'existe pas dans le jeu initial, nous avons dû la créer car aucune des instructions originales ne répondait à cette fonction.

Nettoyage :

Le pointeur de pile est décrémenté (pop_arg) pour libérer l'espace occupé par l'adresse de retour, finalisant ainsi l'outro de la fonction.

Cross Compilation :

Nous avons définie les registres par 3 valeurs :

- mem adr : représente l'adresse réel de la valeur du registre dans la RAM.
- unstored : permet de savoir si la valeur est différente de celle dans la RAM. Elle indique donc si la valeur du registre doit être store.
- last used : un compteur pour savoir quand le registre a été utilisé la dernière fois. Cet attribut est utilisé dans le cadre où les registres suivent LRU.

Lecture fichier assembleur :

Pour construire le fichier assembleur orienté registre on relit le fichier créé dans la partie précédente. Pour parser le fichier nous avons utilisé une astuce décrite dans le fichier Lex afin de différencier les instructions des arguments.

Allocation :

Nous avons opté pour deux stratégies d'allocation des registres.

- Lecture mémoire : on utilise `allouer_reg`. On regarde si on est en cache-hit. Dans ce cas là, on retourne l'index du registre. Sinon on regarde si on a un registre libre ou on applique LRU. On fait attention à store en cas de registre unstored. On load alors la valeur dans le registre libre.
- Écriture sur destination : on utilise `allouer_reg_destination`. Il s'agit du même fonctionnement que ci-dessus mais sans le load car il est nécessaire d'écrire dans mémoire. On place directement le registre en unstored.

Gestion jump :

A chaque jump, on store les registres qui doivent être stores puis on invalide tous les registres. On invalide les registres car après un jump on ne connaît pas l'état des variables. Cela permet de repartir sur une base de registre propre.

Partie Processeur :

Pour la partie processeur nous avons suivi une implémentation très classique.

ALU :

Pour ce composant, nous avons dans un premier temps créé des variables de 16 bits pour réaliser les calculs sur elles sans risques d'overflow. Grâce à cela, nous pouvons aisément déterminer les différents signaux "flags" comme la retenue ou la présence d'un

overflow. Ces calculs sont faits au sein d'un processus non clocké à l'origine, mais devenus 2 processus clockés pour une optimisation que nous détaillerons ultérieurement.

Pipeline:

Pour le pipeline nous avons suivi la structure indiquée dans le projet. La différence est que nous avons créé des composants pour les registres de pipeline (LIDI, DIEX, EXMEM, MEMRE). Chaque composant réécrit son entrée sur la sortie de manière clockée. Seul LIDI réalise un découpage de l'entrée pour avoir les bonnes sorties (A, B, C et OP).

Multiplexer et LC :

Les multiplexeurs et Logic Controller sont non clockés.

Data memory :

Nous avons implémenté la mémoire de données comme demandé mais avec un changement majeur : La lecture est asynchrone. Cela permet d'éviter d'introduire une bulle et de rendre la sortie directement accessible. Ainsi, nous n'avons pas besoin d'attendre deux cycles pour lire une adresse.

Banc de registres :

Le banc de registre implémente un bypass sur Qa et Qb. Ainsi, lorsque nous sommes en mode W (Write) et que l'adresse de lecture est Qa ou Qb alors le signal de sortie est activé avec la valeur écrite dans le registre.

Choix des instructions :

Signification des bits de poids fort (Code)

- **Bit de poids fort (Bit 7)** : Signale l'utilisation du banc de registre pour le stockage de résultats.
- **Second bit de poids fort (Bit 6)** : Signale l'utilisation de l'ALU.
- **Troisième bit de poids fort (Bit 5)** : Signale l'utilisation de la mémoire de données.
- **Quatrième bit de poids fort (Bit 4)** : Signale l'utilisation du banc de registre en lecture.

Opération	Code	Format d'instruction				Description
		OP	A	B	C	
Addition	0xD1	ADD	Ri	Rj	Rk	$[Ri] \leftarrow [Rj] + [Rk]$
Multiplication	0xD2	MUL	Ri	Rj	Rk	$[Ri] \leftarrow [Rj] * [Rk]$
Soustraction	0xD3	SOU	Ri	Rj	Rk	$[Ri] \leftarrow [Rj] - [Rk]$
Division	0xD4	DIV	Ri	Rj	Rk	$[Ri] \leftarrow [Rj] / [Rk]$
Copie	0x95	COP	Ri	Rj	--	$[Ri] \leftarrow [Rj]$
Affectation	0x86	AFC	Ri	j	--	$[Ri] \leftarrow j$
Chargement	0xA7	LOAD	Ri	@j	--	$[Ri] \leftarrow [@j]$
Sauvegarde	0x38	STORE	@i	Rj	--	$[@i] \leftarrow [Rj]$
INSTRUCTIONS RAJOUTÉES						
Saut incond.	0x09	JMP			--	
Saut si false	0x0A	JMF				
charg. par déréférenceme nt	0xAB	LOADR	Ri	Rj		

Chemin critique :

Lors de la synthétisation de notre solution, nous arrivions à un worst négative slack aux alentours de 1500 ns. C'est un très mauvais résultat par rapport à la fréquence visée. En analysant le chemin critique via les outils de Vivado nous avons pu observer le tableau suivant :

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock
↳ Path 10	1446.445	88	23	89	di_ex/C_OUT_reg[0]/C	ex_memv/B_OUT_reg[5]_rep_0/D	53.523	22.609	30.914	1500.0	sysclk_pin	sysclk_pin
↳ Path 9	1446.431	88	23	89	di_ex/C_OUT_reg[0]/C	ex_memv/B_OUT_reg[5]_rep/D	53.538	22.609	30.929	1500.0	sysclk_pin	sysclk_pin
↳ Path 8	1446.355	87	23	89	di_ex/C_OUT_reg[0]/C	ex_memv/B_OUT_reg[1]_rep_0/D	53.612	22.289	31.323	1500.0	sysclk_pin	sysclk_pin
↳ Path 7	1446.202	88	23	89	di_ex/C_OUT_reg[0]/C	ex_memv/B_OUT_reg[5]/D	53.764	22.609	31.155	1500.0	sysclk_pin	sysclk_pin
↳ Path 6	1446.132	87	23	89	di_ex/C_OUT_reg[0]/C	ex_memv/B_OUT_reg[3]_rep/D	53.882	22.394	31.488	1500.0	sysclk_pin	sysclk_pin
↳ Path 5	1446.120	87	23	89	di_ex/C_OUT_reg[0]/C	ex_memv/B_OUT_reg[3]_rep_0/D	53.890	22.394	31.496	1500.0	sysclk_pin	sysclk_pin
↳ Path 4	1445.890	87	23	89	di_ex/C_OUT_reg[0]/C	ex_memv/B_OUT_reg[2]/D	54.071	22.642	31.429	1500.0	sysclk_pin	sysclk_pin
↳ Path 3	1445.835	87	23	89	di_ex/C_OUT_reg[0]/C	ex_memv/B_OUT_reg[2]_rep_0/D	54.131	22.642	31.489	1500.0	sysclk_pin	sysclk_pin
↳ Path 2	1445.695	87	23	89	di_ex/C_OUT_reg[0]/C	ex_memv/B_OUT_reg[2]_rep/D	54.265	22.642	31.623	1500.0	sysclk_pin	sysclk_pin
↳ Path 1	1445.685	87	24	89	di_ex/C_OUT_reg[0]/C	ex_memv/B_OUT_reg[1]/D	53.996	22.317	31.679	1500.0	sysclk_pin	sysclk_pin

Celui-ci nous indique que ce retard se crée entre DIEX et EXMEM. L'ALU est donc le chemin combinatoire responsable. Nous avons alors tenté de la diviser en 2 étages pour réduire la charge combinatoire entre deux éléments clockés (ici le banc de registres des étages précédents et suivants du pipeline). Un premier qui réalise les calculs. Puis un second qui met à jour les flags. Pour cela, nous avons créé des signaux qui représentent des registres dans l'ALU.

Ainsi, l'ALU est constituée de deux processus réalisés en parallèle. Pour synchroniser ce nouvel étage au reste, nous avons dû créer du retard sur les signaux vers EXMEM. Nous avons dû adapter le reste du pipeline autour avec un composant pour créer un retard artificiel de 2 cycles dans le cas où le multiplexeur viendrait à choisir le signal de bypass autour de l'ALU. En addition de ce composant, les signaux transportant l'opcode (OP) et le premier argument (A) ont dû eux aussi être retardés de 2 cycles mais de manière constante, ce qui était plus simple à implémenter. Finalement, pour éviter les aléas de données, nous devons non plus placer 3 NOP mais 4.

Malgré tous nos efforts, cette création d'étage n'a pas raccourci le worst négative slack. Il a simplement déplacé le transfert le plus long entre DIEX et registre de l'ALU.

Nous voyons bien que le chemin critique est dû à la présence de l'ALU. Tout le retard se fait ici. Nous pensons qu'il vient de la présence de toutes les opérations combinatoires sur un seul étage mais nous n'avons pas eu le temps d'implémenter une séparation des calculs plus intelligente, comme mettre la moitié des opérations possibles sur le premier étage et l'autre sur le second étage de l'ALU.

LLM usage : We used a LLM to help us through specific problems such as hidden syntax errors. We used it as a helper and not to copy paste huge stacks of code. We also used IA to create tests files for the VHDL part (ALU, ROM hard code, DATA_MEM).